REPORT

Reviewer report: encode/httpx#3757: Fix MockTransport

not setting elapsed property

Run httpx-3757, tested 2026-11-14 11:32 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-14 12:56 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

VERIFIED

Recommended action: Approve with notes Confidence: Medium

The PR's tests behave the same or better than on main, in all 3 runs per side. Some checks gave warnings: No

flaky tests

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Fix MockTransport not setting elapsed property (encode/httpx#3757).

• What we observed: 19 flaky test(s): tests.client.test_client::test_base_url,

tests.client.test_client::test_build_post_request, tests.client.test_client::test_build_request,

tests.client.test_client::test_cannot_stream_async_request, tests.client.test_client::test_delete (+14); 114 test(s)

ran on main, 117 on the PR; 3 new test(s) pass, 94 unchanged (1 of them skipped on both sides); 2 test failure(s)

pre-existing and identical on both sides; flaky test(s) shown only: their result changes between runs of the same

side, so they do not decide the verdict; probe: claim(s) C1, C2, C3, C4 proven.

• What you should do: You can approve; read the notes under 'What went wrong' first.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 7.7 GB (of 15.6 GB).

• Setup check: free disk 92.7 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: httpx python setup install instructions https://github.com/encode/httpx site:github.com/encode/httpx

• Query: httpx python documentation install https://github.com/encode/httpx site:github.com/encode/httpx

• Query: httpx python install requirements https://pypi.org/project/httpx/ https://github.com/encode/httpx

site:pypi.org

• Source: httpx — https://pypi.org/project/httpx/

• Source: GitHub - encode/httpx: A next generation HTTP client for Python. ■ · GitHub —

https://github.com/encode/httpx

• Source: Version 1.0 public call. ■■ · encode/httpx · Discussion #3344 · GitHub —

https://github.com/encode/httpx/discussions/3344

• Source: Installation Error · encode/httpx · Discussion #3119 · GitHub —

https://github.com/encode/httpx/discussions/3119

• Source: httpx/docs/contributing.md at master · encode/httpx · GitHub —

https://github.com/encode/httpx/blob/master/docs/contributing.md

• Setup plan: the same packages on main and pr.

• Setup plan: certifi is noted, installed with the project.

• Setup plan: httpcore is noted, installed with the project.

• Setup plan: anyio is noted, installed with the project.

• Setup plan: idna is noted, installed with the project.

• Setup plan: install chardet==5.2.0 (listed in requirements.txt) on main and pr.

• Setup plan: install mkdocs==1.6.1 (listed in requirements.txt) on main and pr.

• Setup plan: install mkautodoc==0.2.0 (listed in requirements.txt) on main and pr.

• Setup plan: install mkdocs-material==9.6.18 (listed in requirements.txt) on main and pr.

• Setup plan: install build==1.3.0 (listed in requirements.txt) on main and pr.

• Setup plan: install twine==6.1.0 (listed in requirements.txt) on main and pr.

• Setup plan: install coverage[toml]==7.10.6 (listed in requirements.txt) on main and pr.

• Setup plan: install cryptography==45.0.7 (listed in requirements.txt) on main and pr.

• Setup plan: install mypy==1.17.1 (listed in requirements.txt) on main and pr.

• Setup plan: install pytest==8.4.1 (listed in requirements.txt) on main and pr.

• Setup plan: install ruff==0.12.11 (listed in requirements.txt) on main and pr.

• Setup plan: install trio==0.31.0 (listed in requirements.txt) on main and pr.

• Setup plan: install trio-typing==0.10.0 (listed in requirements.txt) on main and pr.

• Setup plan: install trustme==1.2.1 (listed in requirements.txt) on main and pr.

• Setup plan: install uvicorn==0.35.0 (listed in requirements.txt) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.9, >=3.8

• Setup plan: refused be: its PyPI page does not link to this GitHub repository

• Setup plan: refused published: its PyPI page does not link to this GitHub repository

• Setup plan: refused under: its PyPI page does not link to this GitHub repository

• Setup plan: refused s: its PyPI page does not link to this GitHub repository

• Installed chardet 5.2.0 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed mkdocs 1.6.1 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed mkautodoc 0.2.0 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed mkdocs-material 9.6.18 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed build 1.3.0 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed twine 6.1.0 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed coverage 7.10.6 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed cryptography 45.0.7 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed mypy 1.17.1 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed pytest 8.4.1 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed ruff 0.12.11 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed trio 0.31.0 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed trio-typing 0.10.0 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed trustme 1.2.1 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed uvicorn 0.35.0 from requirements.txt (listed in requirements.txt) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (96 unchanged, 3 runs per side) | pytest-pr-1.log:1864 «1 failed, 101 passed, 1

skipped, 18 errors in 122.04s (0:02:02)»

| P-C1 | Before the fix, accessing response.elapsed on MockTransport responses raises a RuntimeError. (PR

description) | proven | probe model-p1, c1.sync_elapsed_access: value: "RuntimeError" -> "0:00:00" |

probe-model-p1-pr-1.jsonl:1 «"observation": "c1.sync_elapsed_access"»

| P-C2 | After the fix, MockTransport automatically sets response.elapsed to timedelta(0) for both sync and async

requests. (PR description) | proven | probe model-p1, c2.sync_elapsed_value: value: null -> "0:00:00" |

probe-model-p1-pr-1.jsonl:2 «"observation": "c2.sync_elapsed_value"»

| P-C3 | After the fix, accessing response.elapsed returns timedelta(0) without raising an error. (PR description) |

proven | probe model-p1, c3.sync_elapsed_ok: value: false -> true | probe-model-p1-pr-1.jsonl:3 «"observation":

"c3.sync_elapsed_ok"»

| P-C4 | After the fix, users no longer need to manually set resp.elapsed = timedelta(0) in their handler functions.

(PR description) | proven | probe model-p1, c4.sync_no_boilerplate: value: false -> true |

probe-model-p1-pr-1.jsonl:4 «"observation": "c4.sync_no_boilerplate"»

| P-C1 | Before the fix, accessing response.elapsed on MockTransport responses raises a RuntimeError. (PR

description) | proven | probe model-p2, c1.async_elapsed_access: value: "RuntimeError" -> "0:00:00" |

probe-model-p2-pr-1.jsonl:1 «"observation": "c1.async_elapsed_access"»

| P-C2 | After the fix, MockTransport automatically sets response.elapsed to timedelta(0) for both sync and async

requests. (PR description) | proven | probe model-p2, c2.async_elapsed_value: value: null -> "0:00:00" |

probe-model-p2-pr-1.jsonl:2 «"observation": "c2.async_elapsed_value"»

| P-C3 | After the fix, accessing response.elapsed returns timedelta(0) without raising an error. (PR description) |

proven | probe model-p2, c3.async_elapsed_ok: value: false -> true | probe-model-p2-pr-1.jsonl:3

«"observation": "c3.async_elapsed_ok"»

| P-C4 | After the fix, users no longer need to manually set resp.elapsed = timedelta(0) in their handler functions.

(PR description) | proven | probe model-p2, c4.async_no_boilerplate: value: false -> true |

probe-model-p2-pr-1.jsonl:4 «"observation": "c4.async_no_boilerplate"»

Citation check (PASS): 9 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 114 / 117 | -

| Tests unchanged | 96 | | New tests that pass | 3 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 19 |

tests.client.test_client::test_base_url, tests.client.test_client::test_build_post_request,

tests.client.test_client::test_build_request,
tests.client.test_client::test_cannot_stream_async_request,

tests.client.test_client::test_delete

| Tests that fail on main and pass on the PR | 0 | | Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 2 |

tests.client.test_client::test_raw_client_header, tests.client.test_client::test_server_extensions

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-p1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p1.py, frozen 3c0d34203698

at 2026-11-14T06:05:33.142987+00:00): Test synchronous MockTransport elapsed handling.

• P-C1 (PR description): Before the fix, accessing response.elapsed on MockTransport responses raises a

RuntimeError. Expects the covered observations to differ.

• P-C2 (PR description): After the fix, MockTransport automatically sets response.elapsed to timedelta(0) for both

sync and async requests. Expects the covered observations to differ.

• P-C3 (PR description): After the fix, accessing response.elapsed returns timedelta(0) without raising an error.

Expects the covered observations to differ.

• P-C4 (PR description): After the fix, users no longer need to manually set resp.elapsed = timedelta(0) in their

handler functions. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.sync_elapsed_access | P-C1 | as claimed | value: "RuntimeError" -> "0:00:00" |

probe-model-p1-main-1.jsonl:1 «"observation": "c1.sync_elapsed_access"»
probe-model-p1-pr-1.jsonl:1

«"observation": "c1.sync_elapsed_access"»

| c2.sync_elapsed_value | P-C2 | as claimed | value: null -> "0:00:00" | probe-model-p1-main-1.jsonl:2

«"observation": "c2.sync_elapsed_value"»
probe-model-p1-pr-1.jsonl:2 «"observation":

"c2.sync_elapsed_value"»

| c3.sync_elapsed_ok | P-C3 | as claimed | value: false -> true | probe-model-p1-main-1.jsonl:3 «"observation":

"c3.sync_elapsed_ok"»
probe-model-p1-pr-1.jsonl:3 «"observation": "c3.sync_elapsed_ok"»

| c4.sync_no_boilerplate | P-C4 | as claimed | value: false -> true | probe-model-p1-main-1.jsonl:4 «"observation":

"c4.sync_no_boilerplate"»
probe-model-p1-pr-1.jsonl:4 «"observation": "c4.sync_no_boilerplate"»

model-p2 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p2.py, frozen 38ddc5990229

at 2026-11-14T06:05:33.143145+00:00): Test asynchronous MockTransport elapsed handling.

• P-C1 (PR description): Before the fix, accessing response.elapsed on MockTransport responses raises a

RuntimeError. Expects the covered observations to differ.

• P-C2 (PR description): After the fix, MockTransport automatically sets response.elapsed to timedelta(0) for both

sync and async requests. Expects the covered observations to differ.

• P-C3 (PR description): After the fix, accessing response.elapsed returns timedelta(0) without raising an error.

Expects the covered observations to differ.

• P-C4 (PR description): After the fix, users no longer need to manually set resp.elapsed = timedelta(0) in their

handler functions. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.async_elapsed_access | P-C1 | as claimed | value: "RuntimeError" -> "0:00:00" |

probe-model-p2-main-1.jsonl:1 «"observation": "c1.async_elapsed_access"»
probe-model-p2-pr-1.jsonl:1

«"observation": "c1.async_elapsed_access"»

| c2.async_elapsed_value | P-C2 | as claimed | value: null -> "0:00:00" | probe-model-p2-main-1.jsonl:2

«"observation": "c2.async_elapsed_value"»
probe-model-p2-pr-1.jsonl:2 «"observation":

"c2.async_elapsed_value"»

| c3.async_elapsed_ok | P-C3 | as claimed | value: false -> true | probe-model-p2-main-1.jsonl:3 «"observation":

"c3.async_elapsed_ok"»
probe-model-p2-pr-1.jsonl:3 «"observation": "c3.async_elapsed_ok"»

| c4.async_no_boilerplate | P-C4 | as claimed | value: false -> true | probe-model-p2-main-1.jsonl:4

«"observation": "c4.async_no_boilerplate"»
probe-model-p2-pr-1.jsonl:4 «"observation":

"c4.async_no_boilerplate"»

Citation check (PASS): 16 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (126.2 s run).

Coverage gate (python, confidence normal): 3 of 3 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| httpx/_transports/mock.py | 3 | 3

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 324e16aab828 of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: pytest exit 1; 2 failure(s)/error(s) pre-existing and identical on both sides.

• Changed code was exercised (coverage gate): 3 of 3 changed executable line(s) ran (100%).

• Probes ran on both sides: 2 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 9 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 16 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| No flaky tests | WARN | 19 test(s) changed result between runs of the same side:

tests.client.test_client::test_base_url, tests.client.test_client::test_build_post_request,

tests.client.test_client::test_build_request,
tests.client.test_client::test_cannot_stream_async_request,

tests.client.test_client::test_delete (+14) | Listed for the record. A flaky test is not a regression and is not

evidence either way.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | pytest exit 1; 2 failure(s)/error(s) pre-existing and identical on both sides

| Changed code was exercised (coverage gate) | PASS | 3 of 3 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 2 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| No flaky tests | WARN | 19 test(s) changed result between runs of the same side:

tests.client.test_client::test_base_url, tests.client.test_client::test_build_post_request,

tests.client.test_client::test_build_request,
tests.client.test_client::test_cannot_stream_async_request,

tests.client.test_client::test_delete (+14)

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 9 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 16 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-14/httpx-3757 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/encode/httpx/pull/3757

Evidence links

• Pull request encode/httpx#3757

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-p1-main-1.jsonl

• probe-model-p1-pr-1.jsonl

• probe-model-p2-main-1.jsonl

• probe-model-p2-pr-1.jsonl

• main run 1 result: pytest-main-1.log:1816 «1 failed, 100 passed, 1 skipped, 16 errors in 122.15s (0:02:02)»

• pr run 1 result: pytest-pr-1.log:1864 «1 failed, 101 passed, 1 skipped, 18 errors in 122.04s (0:02:02)»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-14 12:56 IST